СОДЕРЖАНИЕ

4. Требования 2
4.3. Требования к видам обеспечения 2
4.3.1. Требования к информационному обеспечению 2
4.3.2. Требования к программному обеспечению 7
4.3.3. Требования к техническому обеспечению 17

4. Требования

4.3. Требования к видам обеспечения

Система строится по клиент-серверной архитектуре:

— клиентская часть — клиентское приложение СУДВ. Оно собирается в двух вариантах: мобильное приложение для смартфонов под управлением ОС Android и веб-клиент, который открывается в браузере на рабочем компьютере. Оба варианта собираются из одного кода и выполняют одинаковые функции. Веб-клиент нужен сотрудникам, которые работают за компьютером: устанавливать его не требуется, достаточно открыть в браузере адрес сервера;

— серверная часть — серверное приложение и СУБД PostgreSQL, размещаемые на сервере предприятия. Серверное приложение также передаёт браузеру файлы веб-клиента.

Клиентская и серверная части разрабатываются в одном репозитории на языке Kotlin и состоят из следующих модулей:

— `client-ui` — экраны клиентского приложения;

— `client-data` — обмен данными с сервером и хранение токена доступа;

— `client-domain` — интерфейс `IApiClient`, через который экраны получают данные. Экраны зависят только от него, а не от способа обмена с сервером;

— `android-app` — сборка мобильного приложения;

— `web-app` — сборка веб-клиента;

— `server` — серверное приложение;

— `shared:contract` — контракт взаимодействия клиента и сервера;

— `shared:domain` — общие правила предметной области.

Модули `shared:contract` и `shared:domain` подключаются и к клиентскому, и к серверному приложению.

Далее приведены требования к информационному, программному и техническому обеспечению системы.

4.3.1. Требования к информационному обеспечению

4.3.1.1. Состав и организация данных

Данные системы должны храниться в реляционной базе данных `sudv` под управлением СУБД PostgreSQL 16. Обращаться к БД может только серверное приложение, через компонент «Доступ к данным».

Сами файлы вложений и отчётов хранятся в каталоге `/srv/sudv/files` на сервере, а в БД записываются только сведения о них: имя, тип, размер и ключ хранения (`storage_key`) — имя файла в каталоге. Так БД не разрастается из-за скан-копий.

Состав таблиц БД приведён в таблице (см. Таблица 4.1). Для каждой таблицы указан интерфейс компонента «Доступ к данным», через который с ней работают прикладные компоненты.

Таблица 4.1 — Состав таблиц базы данных

Таблица	**Содержимое**	**Интерфейс доступа**
`departments`	Подразделения предприятия	`IEmployeeRepository`
`employees`	Справочник сотрудников	`IEmployeeRepository`
`users`	Учётные записи пользователей и их роли	`IUserRepository`
`penalties`	Реестр взысканий	`IPenaltyRepository`
`penalty_removals`	Реквизиты приказов о досрочном снятии взысканий	`IPenaltyRepository`
`attachments`	Сведения о прикреплённых файлах	`IPenaltyRepository`
`status_events`	Журнал событий: смены статусов взысканий	`IEventLogRepository`
`reports`	Архив сформированных сводных отчётов	`IReportRepository`
`report_departments`	Подразделения, по которым сформирован отчёт	`IReportRepository`

4.3.1.2. Схема базы данных

Схема БД приведена в нотации «вороньей лапки» (crow’s foot). Условные обозначения приведены на рисунке (см. Рис. 4.1).

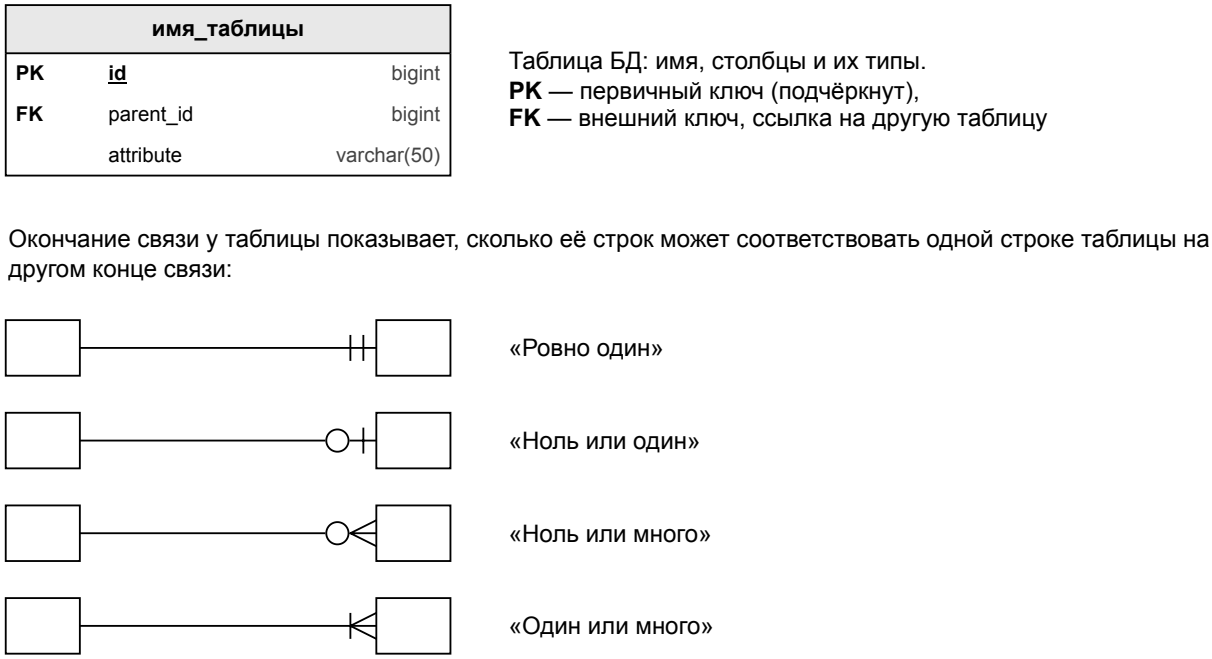


Рисунок 4.1 — Условные обозначения схемы базы данных

Схема БД (см. Рис. 4.2) содержит следующие связи:

— каждый сотрудник относится ровно к одному подразделению, в подразделении может быть сколько угодно сотрудников;
— у сотрудника может быть не более одной учётной записи. Через сотрудника учётная запись связана с подразделением, поэтому руководителю показываются только взыскания его подразделения;
— у сотрудника может быть сколько угодно взысканий;
— у взыскания может быть сколько угодно вложений и записей в журнале событий и не более одной записи о досрочном снятии;
— сводный отчёт формируется по одному или нескольким подразделениям. Связь «многие ко многим» между отчётами и подразделениями хранится в таблице `report_departments`.

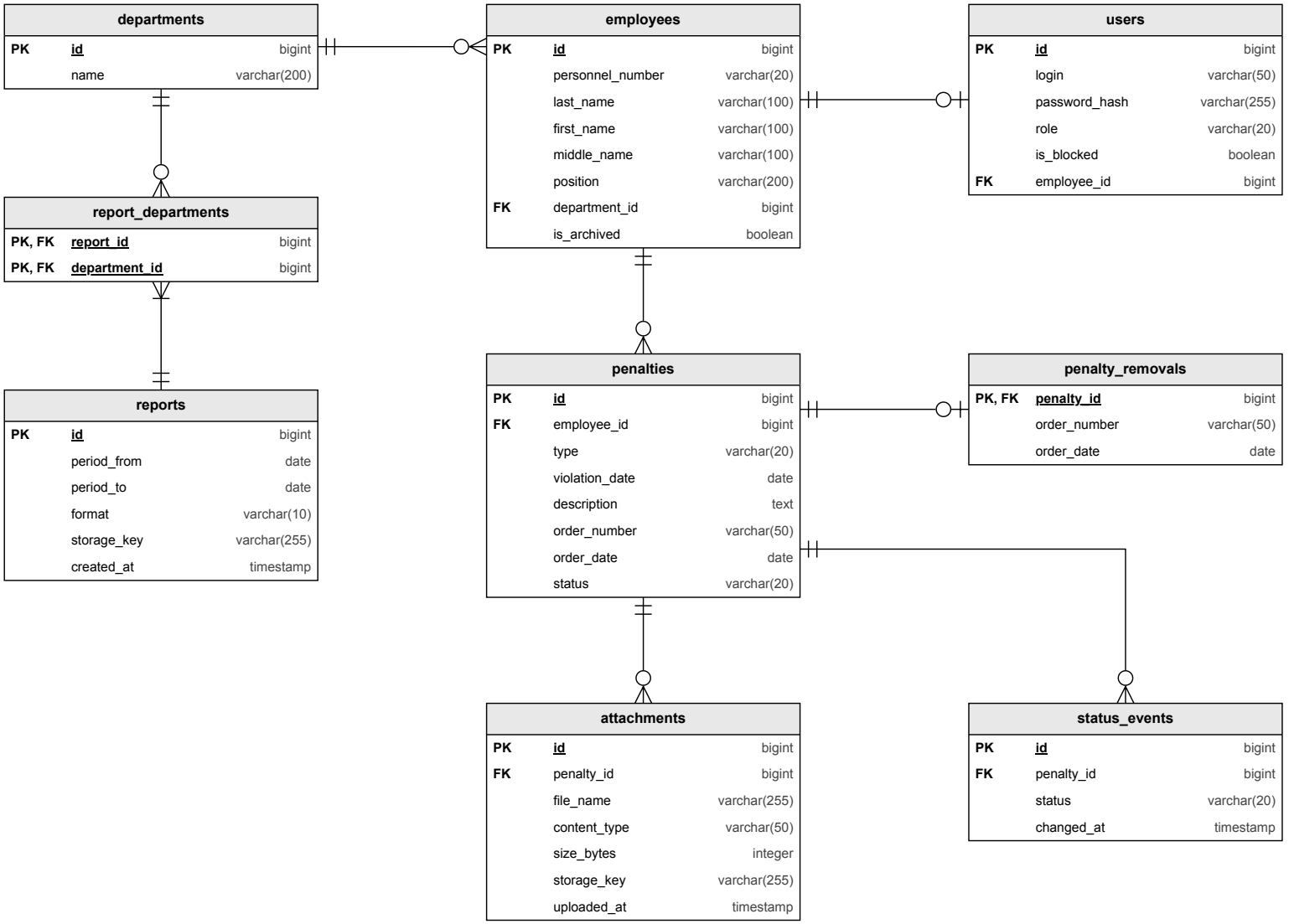


Рисунок 4.2 — Схема базы данных СУДВ

Ограничения целостности данных приведены в таблице (см. Таблица 4.2). Допустимые значения статусов, типов и ролей совпадают с определёнными в модуле `shared:domain`.

Таблица 4.2 — Ограничения целостности данных

Таблица	**Ограничения**
`departments`	`name` уникально
`employees`	`personnel_number` (табельный номер) уникален; `middle_name` может быть пустым. Сотрудники не удаляются, а архивируются (`is_archived`), чтобы сохранить историю взысканий
`users`	`login` и `employee_id` уникальны; `role`: `ADMIN` (администратор), `HR` (кадровик), `MANAGER` (руководитель)
`penalties`	`type`: `REMARK` (замечание), `REPRIMAND` (выговор); `status`: `ACTIVE` («Действует»), `EXPIRED` («Погашено»), `REMOVED` («Снято досрочно»); `order_date` не раньше `violation_date`
`attachments`	`content_type`: `application/pdf` или `image/jpeg`; `size_bytes` больше нуля и не больше установленного лимита
`reports`	`format`: `XLSX` или `CSV`; `period_from` не позже `period_to`

4.3.1.3. Нормализация базы данных

Схема БД приведена к третьей нормальной форме (3НФ).

Первая нормальная форма. Все атрибуты атомарны, повторяющихся групп нет:

— ФИО сотрудника хранится в трёх столбцах: `last_name`, `first_name`, `middle_name`;

— перечень подразделений сводного отчёта не записывается в одно поле через запятую, а хранится отдельными строками в таблице `report_departments`;

— вложения хранятся в отдельной таблице `attachments`, а не в столбцах «файл 1», «файл 2» таблицы взысканий.

Вторая нормальная форма. У каждой таблицы есть первичный ключ. В таблицах с простым ключом (`id`) частичных зависимостей быть не может. Единственная таблица с составным ключом — `report_departments` — неключевых атрибутов не содержит.

Третья нормальная форма. Транзитивных зависимостей нет: неключевые атрибуты зависят только от первичного ключа своей таблицы:

— название подразделения хранится в `departments`, а у сотрудника — только ссылка `department_id`. Если бы название хранилось в `employees`, оно зависело бы от подразделения, а не от сотрудника;

— дата погашения взыскания не хранится, а вычисляется как `order_date` плюс один год. Иначе она зависела бы от неключевого атрибута `order_date`. Оставшийся срок действия также вычисляется.

Реквизиты приказа о досрочном снятии вынесены в таблицу `penalty_removals`, потому что они есть не у каждого взыскания. Так в `penalties` нет столбцов, которые у большинства строк пусты.

4.3.1.4. Принятые допущения

— В архиве хранятся только сводные отчёты по подразделениям. Персональная карточка нарушений формируется по текущим данным при каждом запросе, поэтому в архив не сохраняется.
— Пароли пользователей хранятся только в виде хеша (`password_hash`).

4.3.2. Требования к программному обеспечению

4.3.2.1. Состав программного обеспечения

Перечень программных средств, используемых при создании и эксплуатации системы, приведён в таблице (см. Таблица 4.3). Все средства распространяются по свободным лицензиям. Приобретать покупные программные средства не требуется.

Таблица 4.3 — Перечень программных средств

Программное средство	**Где используется**	**Назначение**
Kotlin	все модули	Язык разработки
Compose Multiplatform	`client-ui`	Пользовательский интерфейс: одни и те же экраны для Android и браузера
Decompose	`client-ui`	Переходы между экранами; в веб-клиенте — адреса страниц и кнопка «Назад» браузера
Ktor	`server`, `client-data`	На сервере — приём HTTP-запросов и аутентификация пользователей, в клиентском приложении — отправка запросов к серверу
DataStore	`client-data`	Хранение токена доступа на устройстве и в браузере
Metro	`server`, клиентские модули	Сборка объектов приложения из модулей (внедрение зависимостей)
Exposed, R2DBC	`server`	Доступ к БД

Программное средство	Где используется	Назначение
PostgreSQL 16	сервер	СУБД
JVM 21	сервер	Среда выполнения серверного приложения
Docker	сервер	Запуск серверного приложения и СУБД в контейнерах
ОС Linux	сервер	Операционная система сервера
ОС Android 8.0 и выше	смартфон	Операционная система мобильного устройства
Браузер с поддержкой WebAssembly GC	рабочий компьютер	Среда выполнения веб-клиента: Chrome 119, Firefox 120, Safari 18.2 и новее

Выбор основных средств разработки обоснован следующим:

— **Kotlin** — один язык для клиентского приложения, сервера и общих модулей. Поэтому модули `shared:contract` и `shared:domain` подключаются к обеим частям системы без дублирования кода;

— **Compose Multiplatform** — библиотека пользовательского интерфейса от JetBrains. Экраны пишутся один раз и работают и на Android, и в браузере (веб-клиент компилируется в WebAssembly). Поэтому веб-клиент не разрабатывается отдельно, и функции двух вариантов клиента не расходятся;

— **Ktor** — фреймворк для Kotlin от компании JetBrains, разработчика языка. Он используется и на сервере, и в клиентском приложении, поэтому обе стороны работают с одним описанием API из `shared:contract`. В приложение подключаются только нужные функции: маршрутизация, преобразование данных в JSON, аутентификация;

— **Exposed** — библиотека доступа к БД для Kotlin от JetBrains. Запросы к БД записываются кодом на Kotlin, поэтому ошибки в именах таблиц и типах полей обнаруживаются при компиляции, а не во время работы системы. К PostgreSQL Exposed подключается через R2DBC — неблокирующий интерфейс доступа к БД: пока БД выполняет запрос, поток сервера не простаивает и обслуживает других пользователей. Ktor тоже неблокирующий, поэтому запрос пользователя обрабатывается без блокировок от начала до конца.

К программному обеспечению предъявляются следующие требования:

— клиентское и серверное приложения должны взаимодействовать только через программный интерфейс `ISudvApi` (REST, формат

JSON). Состав запросов и ответов, адреса методов и коды ошибок описываются один раз, в модуле `shared:contract`;

— правила предметной области (статусы взысканий и допустимые переходы между ними, роли пользователей, правило расчёта даты погашения, правила проверки вводимых данных) должны быть реализованы один раз, в модуле `shared:domain`;

— модули `shared:contract` и `shared:domain` не должны зависеть от Android и от Ktor. Так они одинаково используются на сервере, в мобильном приложении и в браузере;

— модуль `shared:contract` может зависеть от `shared:domain`, но не наоборот. Статусы, типы взысканий и роли определяются один раз, в `shared:domain`, а объекты передачи данных их используют. Правила предметной области при этом не зависят от формата обмена данными;

— модуль `client-ui` не должен зависеть от `client-data`: экраны получают данные только через интерфейс `IApiClient` из модуля `client-domain`, а его реализацию подключают модули сборки `android-app` и `web-app`. Поэтому работа экранов проверяется автоматическими тестами без сервера;

— серверное приложение должно работать в любой ОС, где есть JVM 21, и не должно зависеть от аппаратной платформы сервера.

4.3.2.2. Архитектура программного обеспечения

Архитектура программного обеспечения описана диаграммами компонентов UML. Условные обозначения приведены на рисунке (см. Рис. 4.3).



Рисунок 4.3 — Условные обозначения диаграммы компонентов

Общая диаграмма компонентов (см. Рис. 4.4) показывает две исполняемые части системы, общие модули и СУБД.

Клиентское приложение одинаково устроено в обоих вариантах — мобильном (модуль `android-app`) и веб-клиенте (модуль `web-app`) — и включает два компонента:

— «Пользовательский интерфейс» (модуль `client-ui`) — экраны приложения. Компонент не обращается к серверу напрямую и получает данные через интерфейс `IApiClient` (модуль `client-domain`);

— «Клиент API» (модуль `client-data`) — отправляет запросы к серверу, хранит токен доступа, полученный при входе в систему, и преобразует ответы сервера в объекты модуля `shared:contract`.

Модули `android-app` и `web-app` собственных функций не выполняют: они соединяют компоненты в готовое приложение для своей платформы и задают адрес сервера.

Серверное приложение (модуль `server`) предоставляет интерфейс `ISudvApi`, через который клиентское приложение получает все функции систе-

мы. Данные серверное приложение хранит в БД СУДВ под управлением СУБД PostgreSQL и обращается к ней через R2DBC.

Оба приложения зависят от общих модулей (зависимость «use»):

— `shared:contract` — объекты передачи данных (DTO), адреса методов API и коды ошибок. По нему клиент формирует запросы, а сервер их разбирает;

— `shared:domain` — статусы, роли и правила сроков. Клиентское приложение использует его для проверки данных формы перед отправкой и для расчёта оставшегося срока действия взыскания, сервер — для окончательной проверки данных и смены статусов.

Модуль `shared:contract` сам зависит от `shared:domain`: в объектах передачи данных используются статусы, типы взысканий и роли из `shared:domain`.

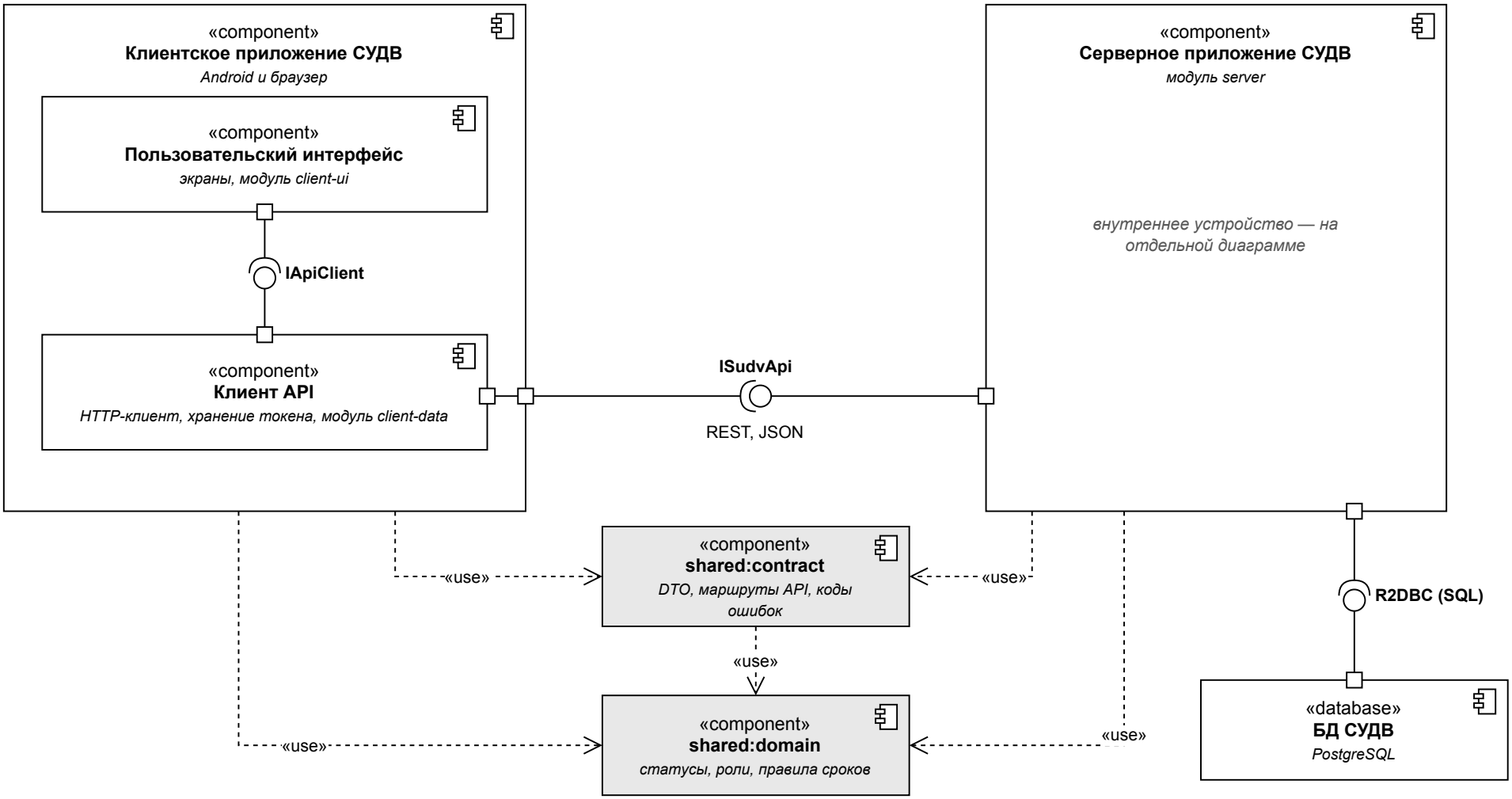


Рисунок 4.4 — Общая диаграмма компонентов СУДВ

4.3.2.3. Состав компонентов серверного приложения

Внутреннее устройство серверного приложения показано на рисунке (см. Рис. 4.5). Компонент «Веб-API» принимает запросы через интерфейс `ISudvApi` и передаёт их прикладным компонентам. Прикладные компоненты соответствуют подсистемам из таблицы «Описание подсистем»:

— «Учёт сотрудников и нарушений» и «Управление доступом» — подсистеме учёта сотрудников и нарушений;
— «Контроль сроков» и «Планировщик заданий» — подсистеме контроля статусов и сроков давности;
— «Отчётность» — подсистеме формирования отчётности.

Компоненты «Доступ к данным» и «Хранилище файлов» — служебные. Сами они функций системы не выполняют, а хранят данные и файлы для прикладных компонентов.

Все запросы к БД собраны в компоненте «Доступ к данным». Прикладные компоненты обращаются к данным только через его интерфейсы и поэтому не зависят от СУБД. Если меняется структура БД, правится только этот компонент. Интерфейсы разделены по видам данных, и каждый прикладной компонент использует только те, данные которых ему нужны:

— `IUserRepository` — учётные записи и роли пользователей;
— `IEmployeeRepository` — сотрудники и подразделения;
— `IPenaltyRepository` — взыскания и сведения о прикреплённых файлах;
— `IReportRepository` — архив сформированных отчётов;
— `IEventLogRepository` — журнал событий (факты смены статусов взысканий).

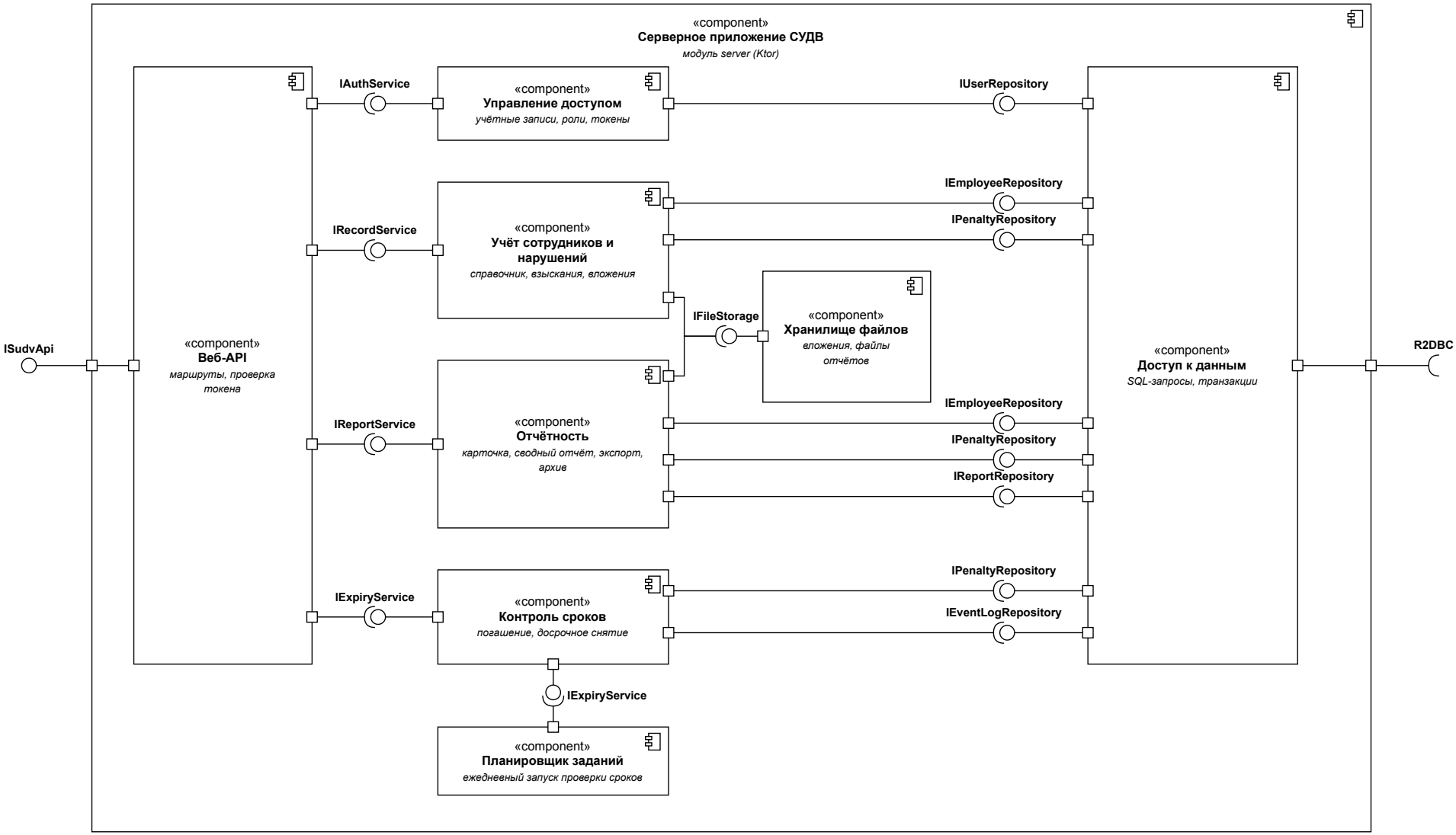


Рисунок 4.5 — Диаграмма компонентов серверного приложения

Ответственность компонентов серверного приложения и их интерфейсы приведены в таблице (см. Таблица 4.4).

Таблица 4.4 — Компоненты серверного приложения

Компонент	**Ответственность**	**Предоставляет**	**Требует**
Веб-API	Приём HTTP-запросов, проверка токена и роли пользователя, преобразование данных по контракту `shared:contract`, передача браузеру файлов веб-клиента	`ISudvApi`	`IAuthService`, `IRecordService`, `IReportService`, `IExpiryService`
Управление доступом	Вход в систему и выдача токенов, ведение учётных записей и ролей, блокировка и разблокировка	`IAuthService`	`IUserRepository`
Учёт сотрудников и нарушений	Ведение справочника сотрудников, регистрация взысканий с фиксацией даты погашения, прикрепление вложений, поиск и фильтрация реестра	`IRecordService`	`IEmployeeRepository`, `IPenaltyRepository`, `IFileStorage`
Отчётность	Формирование персональной карточки и сводного отчёта, экспорт в `.xlsx` и `.csv`, ведение архива отчётов	`IReportService`	`IEmployeeRepository`, `IPenaltyRepository`, `IReportRepository`, `IFileStorage`
Контроль сроков	Автоматическое погашение взысканий с записью в журнал событий, досрочное снятие	`IExpiryService`	`IPenaltyRepository`, `IEventLogRepository`
Планировщик заданий	Ежедневный запуск проверки сроков действия взысканий	—	`IExpiryService`
Хранилище файлов	Проверка формата и размера файлов, сохранение, выдача и	`IFileStorage`	файловая система сервера

Компонент	Ответственность	Предоставляет	Требует
	удаление вложений и файлов отчётов		
Доступ к данным	Чтение и запись данных в БД	`IUserRepository`, `IEmployeeRepository`, `IPenaltyRepository`, `IReportRepository`, `IEventLogRepository`	R2DBC

4.3.2.4. Распределение функций системы по компонентам

Соответствие функций, перечисленных в требованиях к функциям системы, компонентам программного обеспечения приведено в таблице (см. Таблица 4.5). Каждая функция, которую вызывает пользователь, проходит цепочку «Пользовательский интерфейс» → «Клиент API» → «Веб-API». Поэтому в таблице указаны только компоненты, выполняющие саму функцию.

Таблица 4.5 — Распределение функций системы по компонентам

Подсистема	Функция	Компоненты
Подсистема учета сотрудников и нарушений	Ведение справочника сотрудников	Учёт сотрудников и нарушений
	Регистрация дисциплинарного инцидента	Учёт сотрудников и нарушений; правило расчёта даты погашения — `shared:domain`
	Прикрепление сопутствующих материалов	Учёт сотрудников и нарушений, Хранилище файлов
	Управление учетными записями пользователей	Управление доступом
Подсистема контроля статусов и сроков давности	Расчет срока действия взыскания	`shared:domain` (правило расчёта); Учёт сотрудников и нарушений (фиксация даты); Пользовательский интерфейс (оставшийся срок)
	Автоматическое погашение взыскания	Планировщик заданий, Контроль сроков; запись в журнал событий — Доступ к данным (`IEventLogRepository`)
	Досрочное снятие взыскания	Контроль сроков; проверка прав — Веб-API и Управление доступом
	Поиск и фильтрация реестра взысканий	Учёт сотрудников и нарушений

Подсистема	Функция	Компоненты
Подсистема формирования отчетности	Формирование персональной карточки нарушений	Отчётность
	Формирование сводного отчета по подразделениям	Отчётность
	Экспорт отчетных данных	Отчётность, Хранилище файлов
	Ведение архива сформированных отчетов	Отчётность, Хранилище файлов

4.3.3. Требования к техническому обеспечению

4.3.3.1. Состав технических средств

Состав технических средств системы и минимальные требования к их характеристикам приведены в таблице (см. Таблица 4.6).

Таблица 4.6 — Состав технических средств

Техническое средство	Назначение	Требования к характеристикам
Мобильное устройство (смартфон)	Работа пользователей с системой	ОС Android 8.0 и выше; оперативная память не менее 3 ГБ; доступ к сети
Рабочий компьютер	Работа пользователей с системой через веб-клиент	Браузер с поддержкой WebAssembly GC (см. таблицу (см. Таблица 4.3)); оперативная память не менее 4 ГБ; доступ к ЛВС предприятия
Сервер СУДВ	Размещение серверного приложения, БД и файлов вложений	Процессор не менее 4 ядер; оперативная память не менее 8 ГБ; SSD-накопитель не менее 256 ГБ; сетевой адаптер 1 Гбит/с

4.3.3.2. Размещение компонентов системы

Размещение программного обеспечения на технических средствах описано диаграммой развёртывания UML. Условные обозначения приведены на рисунке (см. Рис. 4.6).

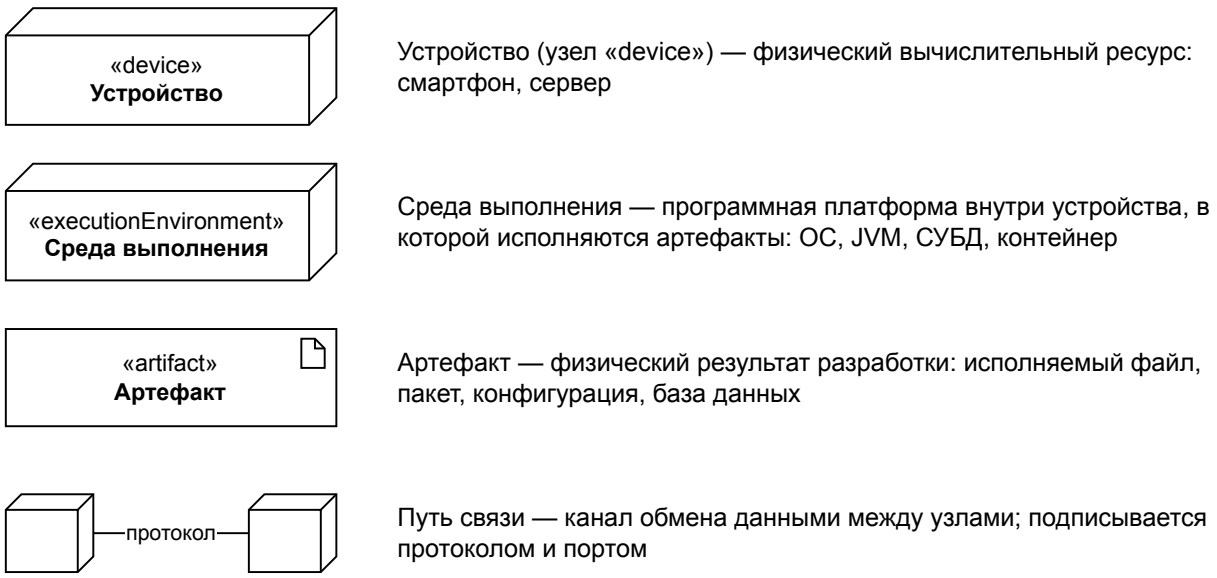


Рисунок 4.6 — Условные обозначения диаграммы развёртывания

Система размещается на трёх видах устройств (см. Рис. 4.7).

1. **Мобильное устройство.** В среде выполнения ОС Android устанавливается артефакт `sudv-android.apk`. Он собирается из модулей `android-app`, `client-ui`, `client-data`, `client-domain`, `shared:contract` и `shared:domain`.
2. **Рабочий компьютер.** Веб-клиент не устанавливается: при открытии адреса сервера браузер загружает с сервера его файлы (основной — `web-app.wasm`) и выполняет их. Веб-клиент собирается из модуля `web-app` и тех же клиентских и общих модулей, что и мобильное приложение.
3. **Сервер СУДВ.** В среде Docker Engine работают два контейнера:
   — контейнер JVM 21 с артефактами `sudv-server.jar` и `web/`. Первый — исполняемый архив серверного приложения, собранный из модулей `server`, `shared:contract` и `shared:domain`. Второй — каталог с файлами веб-клиента, которые серверное приложение передаёт браузеру. Поэтому веб-клиент и API открываются по одному адресу, и обновление сервера сразу обновляет веб-клиент;
   — контейнер PostgreSQL 16 с базой данных `sudv`.

Каталог `/srv/sudv/files` хранит вложения и файлы отчётов. Он находится в файловой системе сервера и подключается к контейнеру JVM как том Docker. Поэтому файлы сохраняются при обновлении и перезапуске контейнеров. Состав контейнеров описан в артефакте `docker-compose.yml`, а настройки — параметры подключения к БД,

ключ подписи токенов, учётная запись первого администратора — в артефакте `.env`. Контейнер JVM получает их как переменные окружения, поэтому один и тот же образ работает на любом сервере.

СУБД не выносится на отдельный компьютер: одного сервера достаточно для числа пользователей предприятия. Порт СУБД доступен только внутри сервера, поэтому к БД может обратиться только серверное приложение.

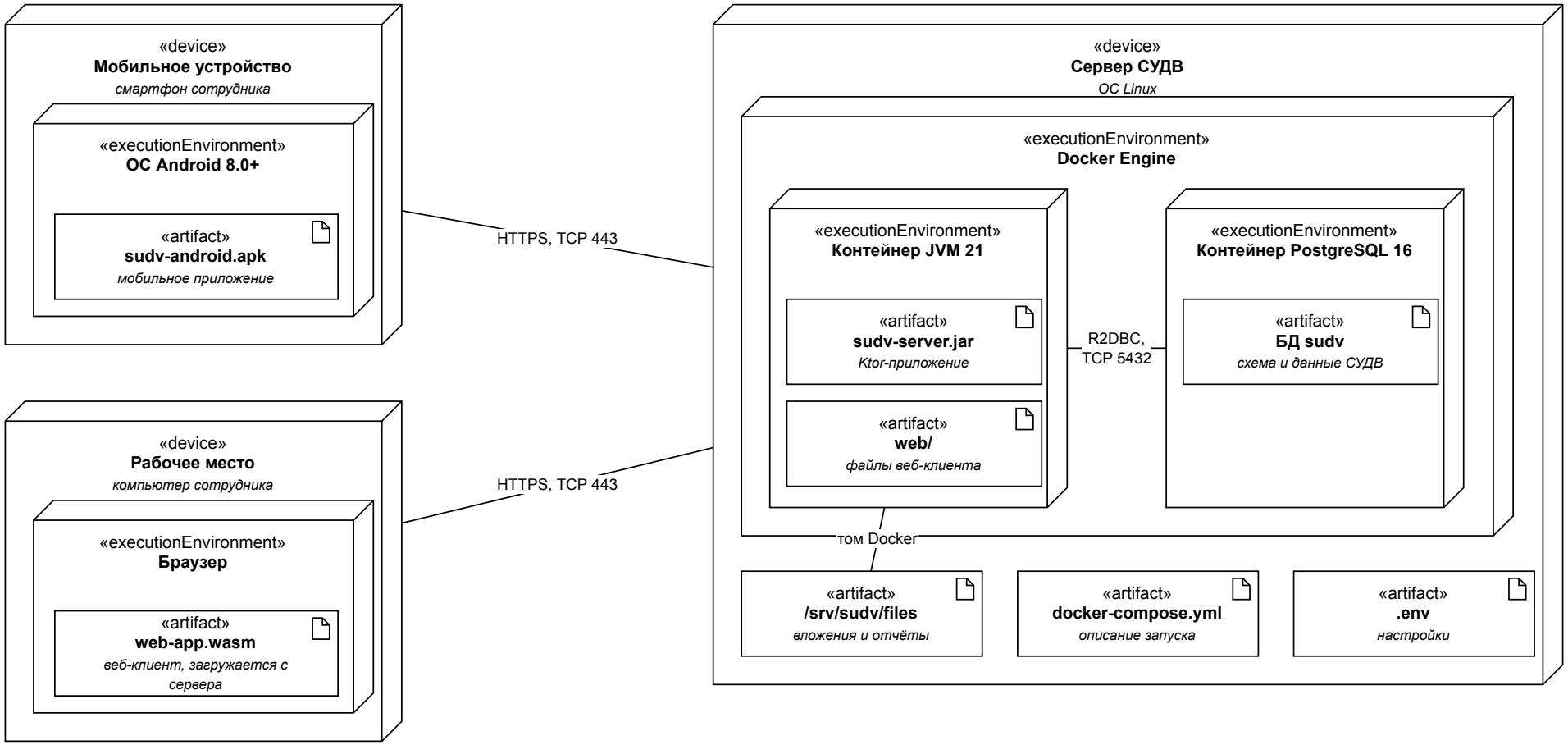


Рисунок 4.7 — Диаграмма развёртывания СУДВ

4.3.3.3. Требования к каналам связи

Пути связи между узлами приведены в таблице (см. Таблица 4.7).

Таблица 4.7 — Пути связи между узлами системы

Путь связи	**Протокол, порт**	**Что передаётся**
Мобильное устройство — сервер СУДВ	HTTPS, TCP 443	Запросы к `ISudvApi` и ответы на них; канал шифруется
Рабочий компьютер — сервер СУДВ	HTTPS, TCP 443	Файлы веб-клиента, запросы к `ISudvApi` и ответы на них; канал шифруется
Контейнер JVM — контейнер PostgreSQL	R2DBC, TCP 5432	SQL-запросы серверного приложения к БД
Контейнер JVM — каталог файлов	Том Docker	Вложения и файлы отчётов

4.3.3.4. Соответствие компонентов и артефактов

Диаграмма компонентов и диаграмма развёртывания согласованы: каждый компонент реализуется артефактом, а каждый артефакт размещён на узле (таблица (см. Таблица 4.8)).

Таблица 4.8 — Соответствие компонентов, артефактов и узлов

Компонент	**Артефакт**	**Размещение**
Пользовательский интерфейс, Клиент API	`sudv-android.apk`; `web-app.wasm` (из каталога `web/`)	Мобильное устройство, ОС Android; рабочий компьютер, браузер
`shared:contract`, `shared:domain`	входят в `sudv-android.apk`, `web-app.wasm` и `sudv-server.jar`	Мобильное устройство; рабочий компьютер; сервер СУДВ
Веб-API, Управление доступом, Учёт сотрудников и нарушений, Отчётность, Контроль сроков, Планировщик заданий, Хранилище файлов, Доступ к данным	`sudv-server.jar`	Сервер СУДВ, контейнер JVM 21
Настройки серверного приложения	`.env`, `docker-compose.yml`	Сервер СУДВ, файловая система
БД СУДВ	БД `sudv`	Сервер СУДВ, контейнер PostgreSQL 16
Данные компонента «Хранилище файлов»	каталог `/srv/sudv/files`	Сервер СУДВ, том Docker

4.3.3.5. Пример взаимодействия компонентов

Согласованность диаграмм проверена на функции «Регистрация дисциплинарного инцидента»:

1. Специалист по кадрам заполняет форму регистрации в мобильном приложении (`sudv-android.apk` на мобильном устройстве). Компонент «Пользовательский интерфейс» проверяет обязательные поля по правилам `shared:domain` и передаёт данные компоненту «Клиент API» через интерфейс `IApiClient`.
2. «Клиент API» формирует запрос по описанию из `shared:contract` и вместе с токеном доступа отправляет его на сервер СУДВ в интерфейс `ISudvApi` (HTTPS, TCP 443).
3. Компонент «Веб-API» в контейнере JVM (`sudv-server.jar`) проверяет токен и роль «Кадровик» через `IAuthService` и передаёт данные в `IRecordService`.
4. Компонент «Учёт сотрудников и нарушений» рассчитывает дату погашения по правилу из `shared:domain`. Затем он сохраняет скан-копии через `IFileStorage` в каталог `/srv/sudv/files`, а запись о взыскании через `IPenaltyRepository` — в БД `sudv` (контейнер PostgreSQL, R2DBC, TCP 5432).
5. Ответ возвращается по той же цепочке. Мобильное приложение показывает карточку взыскания со статусом «Действует».

В веб-клиенте цепочка та же: компоненты «Пользовательский интерфейс» и «Клиент API» выполняются в браузере рабочего компьютера и обращаются к тому же интерфейсу `ISudvApi`.

4.3.3.6. Принятые допущения

— Отдельной программы для администратора нет: администратор работает в том же клиентском приложении, что и остальные пользователи, а доступные ему разделы определяются ролью.

— Резервное копирование БД `sudv` и каталога `/srv/sudv/files` выполняется имеющимися средствами ИТ-отдела, поэтому на диаграмме не показано.